

untangle::executor

Generated by Doxygen 1.18.0

1 executor	1
2 Namespace Index	5
2.1 Namespace List	5
3 Class Index	7
3.1 Class List	7
4 File Index	9
4.1 File List	9
5 Namespace Documentation	11
5.1 untangle Namespace Reference	11
5.1.1 Detailed Description	11
6 Class Documentation	13
6.1 untangle::executor< taskT > Class Template Reference	13
6.1.1 Detailed Description	13
6.1.2 Constructor & Destructor Documentation	14
6.1.2.1 executor()	14
6.1.2.2 ~executor()	14
6.1.3 Member Function Documentation	14
6.1.3.1 add_task()	14
6.1.3.2 stop()	15
6.1.4 Member Data Documentation	15
6.1.4.1 on_task_error	15
7 File Documentation	17
7.1 executor.hpp	17
Index	21

Chapter 1

executor

c++ thread pool executor based on async execution

The interface is one header that exposes a pool of workers, each one an **async** execution in continuous mode, fed by a shared queue of tasks.

This is the prototype, imported. It was written in `async/prototypes/` to answer whether execution is a usable building block for a pool — it is — and it is being hardened here. The behaviour below is what the tests state today, not a settled interface.

requirements

C++23, and the `async` submodule:

```
git submodule update --init --recursive
```

`async` carries its own actuator submodule, which is why the clone has to be recursive. Both are header-only; nothing is built or installed.

The headers report warnings with `std::println`, so they need a standard library that provides `<print>` — GCC 14 or newer.

example

```
#include <executor.hpp>
#include <functional>

int main() {
    untangle::executor<std::function<void(void)>> pool(4);

    for (int i = 0; i < 100; ++i) {
        pool.add_task([i] { work(i); });
    }

    return 0; // ~executor finishes what was submitted, then stops the workers
}
```

the task type

The pool is a template on the task it runs, the way an execution is a template on its action. One pool runs one signature, and the caller names it:

```
untangle::executor<std::function<int(void)>> pool(4); // returns are run and dropped
```

The task type does not have to be a `std::function`. What an execution requires of it is a nested `result_<_type`, which it reads rather than deduces, so any callable declaring one will do — and a pool built on such a callable never touches `std::function::result_type`, which the standard removed in C++20 and both `libc++` and `libstdc++` still supply by grace rather than contract.

```
struct my_task {
    using result_type = void;
    void operator()() const;
};
```

```
untangle::executor<my_task> pool(4);
```

A task type that takes arguments is refused, with a `static_assert` rather than an error inside `std::bind`: the pool queues tasks, and has nowhere to keep arguments for them until a worker frees up.

how work is placed

N executions run in continuous mode behind a shared `std::deque` of tasks.

Nothing overtakes. A task goes straight to a free worker only when the shared queue is empty; otherwise it joins the back of the queue. Finding a free worker does not let a task jump a queue that already has work in it.

Workers pull rather than being pushed to. When one reports its queue finished — on `_finished`, raised by execution the moment its list empties — it takes the front of the shared queue. That is what spreads the work, with no scheduling logic of its own: 400 tasks over 4 workers come out 100/100/100/100.

A worker is asked, not tracked. `execution::is_busy()` is what `add_task()` reads to find a free worker and what the destructor reads to decide the pool is idle. A tally kept here would be the pool's belief about its workers; this is the workers' own answer, and it cannot drift.

lifetime and failure

The destructor finishes what was submitted. It refuses further work, waits for the queue to empty and for every worker to go idle, and only then stops them. A task submitted before the pool goes out of scope has run by the time it does.

`add_task()` says whether the task was taken. It returns false once the pool has stopped accepting — which a task adding more work from inside the pool can see — and false again if the worker it was offered to has stopped, which is what `stop()` leaves behind. A refused task is destroyed rather than queued, so a false answer means it will not run.

`stop()` stops the workers. What they are running they finish; what is still queued stays there unrun, and anything added afterwards is refused. It does not wait for the workers to leave their threads — only the destructor does that, and that wait is what makes destroying the pool safe.

A task that throws does not take the worker with it, and what it threw is not lost. The worker catches it and carries on with the next task; `on_task_error` is where the throw goes:

```

untangle::executor<std::function<void(void)» pool(4);

pool.on_task_error = [](std::exception_ptr thrown) {
    try {
        std::rethrow_exception(thrown);
    } catch (const std::exception& e) {
        log(e.what());
    }
};

```

It carries a `std::exception_ptr` because a task may throw something that is not a `std::exception`, and that is the case most worth hearing about. Assign it before the first `add_task()` — a worker reads it — and expect it on a worker's thread, with more than one worker possibly inside it at once. Leave it unset and the pool warns on `stderr` instead, naming the worker.

A return value is still not collected: a continuous worker keeps none, so a task with something to report carries its own channel.

building the tests

```

cmake -S test -B test/build
cmake --build test/build
ctest --test-dir test/build

```

`test/CMakeLists.txt` fetches googletest at configure time, so the first configure needs network access.

sanitizers

`EXECUTOR_SANITIZE` builds the tests under a sanitizer. It is off by default, because a sanitized build is several times slower and ThreadSanitizer does not ship for every toolchain.

```

cmake -S test -B test/build-asan -DEXECUTOR_SANITIZE=address
cmake --build test/build-asan
ctest --test-dir test/build-asan

```

Accepted values are `address`, `thread`, `undefined`, or `empty`. Anything else is refused at configure time rather than passed through to the compiler.

Use a separate build directory per sanitizer. `address` and `thread` instrument the same accesses in incompatible ways and cannot be combined, so one build directory is one sanitizer.

the reference

`tools/make_doc.sh` builds `doc/refman.pdf` from the header and this README. It needs `doxygen`, `tectonic` and `python3`, and it fails on any `doxygen` warning that is not an `obsolete-tag` notice, so an undocumented member or a broken `\ref` does not reach the reference.

Chapter 2

Namespace Index

2.1 Namespace List

Here is a list of all documented namespaces with brief descriptions:

untangle	A thread pool executor built out of <code>untangle::async::execution</code> objects	11
--------------------------	---	--------------------

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

untangle::executor< taskT >	
Runs tasks on a fixed pool of untangle::async::execution workers	13

Chapter 4

File Index

4.1 File List

Here is a list of all documented files with brief descriptions:

executor.hpp		17
------------------------------	-------	----

Chapter 5

Namespace Documentation

5.1 untangle Namespace Reference

A thread pool executor built out of `untangle::async::execution` objects.

Classes

- class `executor`
Runs tasks on a fixed pool of `untangle::async::execution` workers.

5.1.1 Detailed Description

A thread pool executor built out of `untangle::async::execution` objects.

N workers run in continuous mode behind a shared queue. A task goes straight to a free worker only when the queue is empty; otherwise it joins the back, and a worker takes the front as it frees up.

Chapter 6

Class Documentation

6.1 `untangle::executor< taskT >` Class Template Reference

Runs tasks on a fixed pool of `untangle::async::execution` workers.

```
#include <executor.hpp>
```

Public Member Functions

- `executor` (`std::size_t worker_count`)
Starts `worker_count` workers in continuous mode.
- `~executor` ()
Finishes what has been added, then stops every worker.
- `bool add_task` (`taskT task`)
Queues a task, or hands it to a worker that has nothing to do.
- `std::size_t pending` () `const`
How many tasks are waiting for a worker.
- `void stop` ()
Stops every worker. What they are already running, they finish.

Public Attributes

- `std::function< void(std::exception_ptr)>` `on_task_error`
Called with whatever a task threw. Assigned by the caller.

6.1.1 Detailed Description

```
template<typename taskT>  
class untangle::executor< taskT >
```

Runs tasks on a fixed pool of `untangle::async::execution` workers.

Template Parameters

taskT	The type of the task the pool runs. One pool runs one signature, and a non-void return is run and dropped.
-------	--

6.1.2 Constructor & Destructor Documentation

6.1.2.1 executor()

```
template<typename taskT>
untangle::executor< taskT >::executor (
    std::size_t worker_count)  [inline], [explicit]
```

Starts worker_count workers in continuous mode.

Parameters

worker_count	- How many workers to run. Must not be zero.
--------------	--

Exceptions

std::invalid_argument	- worker_count is zero. Such a pool would take work it could never run.
-----------------------	---

6.1.2.2 ~executor()

```
template<typename taskT>
untangle::executor< taskT >::~~executor ()  [inline]
```

Finishes what has been added, then stops every worker.

Both waits are unbounded, and both report on stderr while they last, naming the workers they are waiting on.

6.1.3 Member Function Documentation

6.1.3.1 add_task()

```
template<typename taskT>
bool untangle::executor< taskT >::add_task (
    taskT task)  [inline]
```

Queues a task, or hands it to a worker that has nothing to do.

Returns

true - the task was accepted. false - it was refused, by a pool that has stopped accepting or by a worker that has stopped. Either way it will not run.

6.1.3.2 `stop()`

```
template<typename taskT>
void untangle::executor< taskT >::stop () [inline]
```

Stops every worker. What they are already running, they finish.

Queued tasks stay unrun and a task added afterwards is refused. Calling it twice is harmless.

Attention

It does not wait for the workers to leave their threads. Only the destructor does that, and that wait is what makes destroying the pool safe.

6.1.4 Member Data Documentation

6.1.4.1 `on_task_error`

```
template<typename taskT>
std::function<void(std::exception_ptr)> untangle::executor< taskT >::on_task_error
```

Called with whatever a task threw. Assigned by the caller.

Nothing else reports a failed task: `add_task()` has answered long before the task runs. Left unset, the pool warns on stderr instead.

Remarks

It receives a `std::exception_ptr`, because a task may throw what is not a `std::exception`. Rethrow it to read it.

Attention

It runs on a worker's thread, and several workers may be in it at once. Assign it before the first `add_task()`, and let nothing escape it.

The documentation for this class was generated from the following file:

- `executor.hpp`

Chapter 7

File Documentation

7.1 executor.hpp

```
00001 // Copyright (c) 2026 Nicolae Popescu. MIT License.
00002
00010 #pragma once
00011
00012 #include <algorithm>
00013 #include <async.hpp>
00014 #include <chrono>
00015 #include <condition_variable>
00016 #include <cstdint>
00017 #include <deque>
00018 #include <exception>
00019 #include <functional>
00020 #include <memory>
00021 #include <mutex>
00022 #include <print>
00023 #include <stdexcept>
00024 #include <string>
00025 #include <thread>
00026 #include <type_traits>
00027 #include <utility>
00028 #include <vector>
00029
00030 namespace untangle {
00031
00038 template <typename taskT>
00039 class executor {
00040     // A task is queued as it stands, with nothing bound to it, so it must take no arguments.
00041     static_assert(std::is_invocable_v<taskT>,
00042         "executor: the task type must be callable with no arguments");
00043
00044 public:
00053     explicit executor(std::size_t worker_count) {
00054         if (worker_count == 0) {
00055             throw std::invalid_argument("executor: worker_count must not be zero");
00056         }
00057
00058         workers_.reserve(worker_count);
00059
00060         for (std::size_t index = 0; index < worker_count; ++index) {
00061             auto next = worker_execution::create_instance("pool_worker_" + std::to_string(index));
00062
00063             // Tells the pool the moment this worker has room for more.
00064             next->on_finished = [this, index] { take_next_task(index); };
00065
00066             // Carries what a task throws to the caller. Assigned here because a worker thread reads it.
00067             next->on_error = [this, name = next->name](std::exception_ptr thrown) {
00068                 report_task_error(name, thrown);
00069             };
00070
00071             // Adds the worker to the poll the destructor waits on.
00072             poll_.add(*next);
00073
00074             workers_.push_back(std::move(next));
00075             workers_.back()->start();
00076         }
00077     }
00078 }
```

```

00085 ~executor() {
00086 {
00087     std::unique_lock<std::mutex> lock(mutex_);
00088     accepting_ = false;
00089
00090     // Waits for the queue to empty and every worker to go idle.
00091     auto interval = std::chrono::milliseconds(report_first_ms);
00092     auto waited = std::chrono::milliseconds(0);
00093
00094     while (!finished_cv_.wait_for(lock, interval,
00095                                     [this] { return pending_.empty() && nothing_running(); })) {
00096         waited += interval;
00097
00098         // Under the lock, so the counts agree with the predicate that just failed.
00099         std::println(
00100             stderr, "executor: still waiting to finish after {}s - {} queued, {} in a task{}",
00101             waited.count() / 1000, pending_.size(), count_workers(&worker_execution::is_busy),
00102             name_workers(&worker_execution::is_busy));
00103
00104         interval = std::min(interval * 2, std::chrono::milliseconds(report_max_ms));
00105     }
00106 }
00107
00108 stop();
00109
00110 // Waits for every worker to leave its thread, which is what makes destroying them safe. A
00111 // different question from the wait above: a worker can be idle and still be in its thread.
00112 auto interval = std::chrono::milliseconds(report_first_ms);
00113 auto waited = std::chrono::milliseconds(0);
00114
00115 while (poll_.is_running()) { // time of check
00116     const auto tick = std::chrono::milliseconds(poll_interval_ms);
00117     std::this_thread::sleep_for(tick);
00118     waited += tick;
00119
00120     if (waited >= interval) {
00121         std::println(stderr, "executor: still waiting to stop after {}s - {} not left its thread{}",
00122                     waited.count() / 1000, count_workers(&worker_execution::is_running),
00123                     name_workers(&worker_execution::is_running));
00124
00125         interval = std::min(interval * 2, std::chrono::milliseconds(report_max_ms));
00126     }
00127 }
00128
00129 // time of use: no worker thread is left to reach workers_.
00130 workers_.clear();
00131 }
00132
00133 bool add_task(taskT task) {
00134     std::lock_guard<std::mutex> lock(mutex_);
00135
00136     if (!accepting_) {
00137         return false;
00138     }
00139
00140     // The queue comes first: a task does not overtake one already waiting in it.
00141     if (pending_.empty()) {
00142         for (std::size_t index = 0; index < workers_.size(); ++index) {
00143             if (!workers_[index]->is_busy()) {
00144                 // A refused task is already destroyed, and stop() stops every worker together, so there
00145                 // is nothing to queue and no other worker to try.
00146                 return give_to_worker(index, std::move(task));
00147             }
00148         }
00149     }
00150
00151     pending_.push_back(std::move(task));
00152     return true;
00153 }
00154
00155 std::size_t pending() const {
00156     std::lock_guard<std::mutex> lock(mutex_);
00157     return pending_.size();
00158 }
00159
00160 void stop() {
00161     // Outside any lock: a worker waking from stop() runs on_finished, which takes mutex_.
00162     for (auto& one : workers_) {
00163         one->stop();
00164     }
00165 }
00166
00167 std::function<void(std::exception_ptr)> on_task_error;
00168
00169 private:
00170 void report_task_error(const std::string& worker, std::exception_ptr thrown) {
00171     if (on_task_error) {

```

```

00207     on_task_error(thrown);
00208     return;
00209 }
00210
00211 try {
00212     std::rethrow_exception(thrown);
00213 } catch (const std::exception& e) {
00214     std::println(stderr, "warning: executor task on '{}' threw: {}", worker, e.what());
00215 } catch (...) {
00216     std::println(stderr, "warning: executor task on '{}' threw an unknown type", worker);
00217 }
00218 }
00219
00220 using worker_execution = untangle::async::execution<taskT>;
00221
00222 [[nodiscard]] bool give_to_worker(std::size_t index, taskT task) {
00223     // add_action() takes the worker's action_mutex under mutex_. The two never cycle, because a
00224     // worker calls on_finished with its action_mutex released.
00225     return workers_[index]->add_action(std::move(task));
00226 }
00227
00228 void take_next_task(std::size_t index) {
00229     std::lock_guard<std::mutex> lock(mutex_);
00230
00231     if (!pending_.empty()) {
00232         taskT next = std::move(pending_.front());
00233         pending_.pop_front();
00234
00235         if (!give_to_worker(index, std::move(next))) {
00236             // The task is already destroyed, so it is reported rather than put back.
00237             std::println(stderr, "executor: {} refused a queued task, which is lost",
00238                 workers_[index]->name);
00239         }
00240     }
00241     return;
00242 }
00243
00244 if (nothing_running()) {
00245     finished_cv_.notify_all();
00246 }
00247
00248 bool nothing_running() const {
00249     for (const auto& one : workers_) {
00250         if (one->is_busy()) {
00251             return false;
00252         }
00253     }
00254     return true;
00255 }
00256
00257 std::size_t count_workers(bool (worker_execution::*state)() const) const {
00258     std::size_t count = 0;
00259
00260     for (const auto& one : workers_) {
00261         if ((one.get()->*state)()) {
00262             ++count;
00263         }
00264     }
00265     return count;
00266 }
00267
00268 std::string name_workers(bool (worker_execution::*state)() const) const {
00269     std::string names;
00270
00271     for (const auto& one : workers_) {
00272         if ((one.get()->*state)()) {
00273             names += names.empty() ? " : " : ", ";
00274             names += one->name;
00275         }
00276     }
00277     return names;
00278 }
00279
00280 static constexpr int poll_interval_ms = 50;
00281
00282 static constexpr int report_first_ms = 1000;
00283 static constexpr int report_max_ms = 30000;
00284
00285 mutable std::mutex mutex_;
00286 std::condition_variable finished_cv_;
00287
00288 async::execution_poll poll_;
00289
00290

```

```
00326  std::deque<taskT> pending_;
00327  std::vector<std::shared_ptr<worker_execution>> workers_;
00328  bool accepting_ = true;
00329 };
00330
00331 } // namespace untangle
```

Index

- ~executor
 - untangle::executor< taskT >, [14](#)
- add_task
 - untangle::executor< taskT >, [14](#)
- executor, [1](#)
 - untangle::executor< taskT >, [14](#)
- on_task_error
 - untangle::executor< taskT >, [15](#)
- stop
 - untangle::executor< taskT >, [14](#)
- untangle, [11](#)
- untangle::executor< taskT >, [13](#)
 - ~executor, [14](#)
 - add_task, [14](#)
 - executor, [14](#)
 - on_task_error, [15](#)
 - stop, [14](#)